



µLab worksheet · a cow of your own

Group _____ Date _____ Time _____

Driver (writes) _____ Mapper (does not write) _____

Hand this to the teacher when the round ends.

1 · WHOSE ACCOUNT IS THIS ON?

The driver's GitHub handle, **one character per box**. Everything you publish today lands under this name, and this sheet is the only thing that connects it back to your group. If we cannot read it, we cannot find your work.

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

The repository you create in step 1 _____

2 · THE FIRST HALF, IN FOUR STEPS

1 ☐ Take the template

Open github.com/oesteban/whalesay and press **Use this template** → **Create a new repository**, under **your own** account. Not a fork.

2 ☐ Branch, and add your cow

`git checkout -b group-N · write group-N.cow · commit · push`. Cows to steal: github.com/paulkaefer/cowsay-files

3 ☐ Build it and make it talk

`docker build -t my_whale .` then `docker run --rm my_whale -f group-N "hi from group N"`

4 ☐ Open the pull request

Against the default branch, **in your own repository**. Then watch the **Actions** tab.

3 · WHAT YOU BUILT

After step 3, run `docker images`.

Size of `my_whale` _____ Size of `alpine` _____

You added one small text file to `alpine` and got a much larger image. What else went in, and which line of the `Dockerfile` put it there?

--

Add the two sizes together. Now check how much disk Docker is really using with `docker system df`. The numbers disagree — why?

--

4 • WHAT `--rm` WAS FOR

Run the same cow twice, then look at what is left behind.

```
docker run --rm my_whale -f group-N "one"
docker run my_whale -f group-N "two" < no --rm
```

docker ps shows _____ docker ps -a shows _____

Both printed a cow and both finished. One of them is still on your machine. What is still there, and what is it still holding?

Run the no- `--rm` version three more times. How many entries in `docker ps -a` now? _____ Clear them, and write the command you used.

5 • A CONTAINER YOU CAN WALK BACK INTO

`docker run -d` detaches. Try it on `my_whale` as it is, first, and see what `docker ps` says a few seconds later.

Now give it something that does not finish immediately:

```
docker run -d --name cow --entrypoint sleep my_whale 600
docker exec cow ls /usr/local/share/cows/
docker exec cow cowsay -f group-N "from inside"
```

Why did the first attempt need no `--entrypoint` and this one does?

Compare `docker exec cow whoami` with `whoami` in your own terminal, and `docker exec cow ls /` with `ls /`. Same machine or not? Say how you know.

Stop it and remove it. Commands used: _____

6 • WHAT THE IMAGE REMEMBERS

Run `docker image history my_whale`.

Lines your Dockerfile added _____ Largest single line _____

Most lines are 0B. What does a line of 0B mean actually happened?

Add a second cow file and rebuild with the same command. Watch the output.

Which steps re-ran, and which said `CACHED`? Write the first step that re-ran. _____

You changed one file. Why did that step, and not an earlier one, have to run again? Guess, and say what would test your guess.

7 • SOMETHING ELSE BUILT IT

After step 4, open the **Actions** tab on your repository.

The run took _____ It ran on _____

That machine has never seen your laptop. What did it need from you to build your image, and where did it get it?

The run finished green, and nothing was published. Say where you looked to be sure of that.

8 • A WEB FRONT END AND A COW BACK END

Two containers. `cow` writes a page and stops. `web` serves it and stays up. They never talk to each other: they share one **named volume**, mounted at a different path in each. Put this in `compose.yaml`:

```
services:
  cow:
    image: my_whale
    entrypoint: ["/bin/sh", "-c"]
    command: [{"echo '<pre>'; cowsay -f group-N 'your words'; echo '</pre>'; } > /srv/index.html"]
    volumes:
      - site:/srv

  web:
    image: nginx:alpine
    depends_on:
      cow:
        condition: service_completed_successfully
    ports: ["8080:80"]
    volumes:
      - site:/usr/share/nginx/html:ro

volumes:
  site:
```

Four things in there are worth knowing rather than copying. `nginx` serves whatever is in `/usr/share/nginx/html`. `cowsay` writes to the screen, not to a file, so it needs a shell to redirect it — that is what overriding the `entrypoint` buys. The `<pre>` stops a browser collapsing the artwork. And `service_completed_successfully` is what stops `web` starting before there is anything to serve.

`docker compose up -d` then `curl localhost:8080`

`docker compose ps` does not list `cow`. Where did it go, and which flag from box 4 finds it? _____

`cow` exited. Is that a failure? Say what would make it one.

`web` never asks `cow` for anything. Trace how the words get from one container to the other, naming each thing they pass through.

Change the words, and run `docker compose up -d` again. Compose recreated one container and left the other alone. Which, and why that one?

Done when `curl localhost:8080` prints your cow, `cow` has exited 0, and `web` is still up.